

TempMail Backend - API Documentation

Overview

TempMail is a disposable/temporary email service backend built with Node.js and Express. It provides temporary email addresses that automatically expire after 1 hour. The system includes an SMTP server to receive emails, WebSocket support for real-time notifications, and a REST API for frontend integration.

Tech Stack

Technology	Purpose
Node.js + Express	HTTP API Server
MongoDB / PostgreSQL	Database (configurable)
Socket.io	Real-time WebSocket notifications
smtp-server	SMTP server to receive emails
mailparser	Parse incoming email data
express-rate-limit	API rate limiting

Project Structure

```
TempMailProj/
├── src/
│   ├── server.js # Entry point - starts HTTP, SMTP, WebSocket
│   ├── app.js # Express app configuration & middleware
│   ├── config/
│   │   ├── db.js # Database connection (MongoDB/PostgreSQL)
│   │   └── initDb.js # Database initialization (tables/indexes)
│   ├── controllers/
│   │   ├── emailController.js # Email endpoint handlers
│   │   └── inboxController.js # Inbox endpoint handlers
│   ├── services/
│   │   ├── emailService.js # Email business logic + rate limiting
│   │   └── inboxService.js # Inbox business logic
│   └── repositories/
│       └── emailRepository.js # Email database operations
```

```
| | └─ inboxRepository.js # Inbox database operations
| └─ routes/
| | └─ emailRoutes.js # API route definitions
| └─ smtp/
| | └─ smtpServer.js # SMTP server for receiving emails
| └─ websocket/
| | └─ socketManager.js # WebSocket (Socket.io) manager
| └─ jobs/
| | └─ cleanupJob.js # Periodic cleanup of expired emails
| └─ utils/
| └─ generateEmail.js # Random email address generator
└─ .env # Environment configuration
└─ package.json
└─ sendTest.js # Test script to send emails locally
```

Environment Variables (.env)

```
env
```

Database Type: "mongodb" or "postgres"

```
DB_TYPE=mongodb
```

MongoDB Configuration

```
MONGO_URI=mongodb+srv://user:pass@cluster.mongodb.net/?retryWrites=true&w=majority
```

```
DB_NAME=temppmail
```

PostgreSQL Configuration (used when DB_TYPE=postgres)

```
DB_HOST=localhost
```

```
DB_PORT=5432
```

```
DB_USER=postgres
```

```
DB_PASSWORD=postgres
```

HTTP Server

```
PORT=3000
```

SMTP Server

SMTP_PORT=2525

MAIL_DOMAIN=tempmail.dev

CORS

CORS_ORIGIN=http://localhost:5173

API Endpoints

1. Generate / Get Email

POST /email/get

Assigns a temporary email to the session. If the session already has an active email, returns the existing one.

Request Body: `json`

```
{
  "session_id": "uuid-string"
}
```

Success Response (200): `json`

```
{
  "success": true,
  "email": {
    "email_address": "abcd7k2m9x@tempmail.dev",
    "status": "ASSIGNED",
    "session_id": "uuid-string",
    "expires_at": "2024-01-15T11:00:00.000Z",
    "created_at": "2024-01-15T10:00:00.000Z"
  }
}
```

Error Response (429 - Rate Limited): `json`

```
{
  "error": "Rate limit exceeded. You can generate max 3 emails per minute. Try again in 45 seconds."
}
```

2. Change Email

POST /email/change

Releases the current email (marks as expired, clears inbox) and assigns a new one.

Request Body: `json`

```
{
  "session_id": "uuid-string"
}
```

Success Response (200): `json`

```
{
  "success": true,
  "email": {
```

```
"email_address": "newx8p3q2r@tempmail.dev",
"status": "ASSIGNED",
"session_id": "uuid-string",
"expires_at": "2024-01-15T12:00:00.000Z",
"created_at": "2024-01-15T11:00:00.000Z"
}
}
```

```
` Error Response (429 - Rate Limited): ` json
{
  "error": "Rate limit exceeded. You can generate max 3 emails per minute. Try again in 30 seconds."
}
`
```

3. Get Email Status

GET /email/status?session_id=uuid-string

Returns the current status of the session's email including remaining time.

Success Response (200): ` json

```
{
  "success": true,
  "email_address": "abcd7k2m9x@tempmail.dev",
  "status": "ASSIGNED",
  "created_at": "2024-01-15T10:00:00.000Z",
  "expires_at": "2024-01-15T11:00:00.000Z",
  "remaining_minutes": 45,
  "is_expired": false
}
```

```
` Error Response (404): ` json
{
  "error": "No active email for this session"
}
`
```

4. Get Inbox

GET /inbox?email=abcd7k2m9x@tempmail.dev

Fetches all received messages for the given email address.

Success Response (200): ` json

```
{
  "success": true,
  "messages": [
    {
      "email_address": "abcd7k2m9x@tempmail.dev",
      "sender": "noreply@example.com",
      "subject": "Welcome!",
      "body": "Hello, thanks for signing up.",
      "received_at": "2024-01-15T10:05:00.000Z"
    }
  ]
}
```

```
]
```

```
}
```

```
`
```

5. Delete Inbox

DELETE /inbox

Clears all messages for the specified email. Only affects that user's data.

Request Body: `json`

```
{
  "email": "abcd7k2m9x@tempmail.dev"
}
```

Success Response (200): `json`

```
{
  "success": true,
  "message": "Inbox cleared"
}
```

```
`
```

6. Health Check

GET /health **Response (200):** `json`

```
{
  "status": "ok",
  "timestamp": "2024-01-15T10:00:00.000Z"
}
```

```
`
```

Rate Limiting

Type	Limit	Scope
API Rate Limit	100 requests / 15 minutes	Per IP address
Email Generation	3 emails / 1 minute	Per session

Email Generation Format

```
`
```

Emails follow the pattern: [4 letters][6 alphanumeric]@domain

```
`
```

- First 4 characters: lowercase letters only (a-z)
- Next 6 characters: mix of letters and numbers (a-z, 0-9)

```
`
```

```
`
```

- Domain: configurable via `MAIL_DOMAIN` env variable

Examples:

- `abcd7k2m9x@tempmail.dev`

- `xkrm4b8w2q@tempmail.dev`

WebSocket Events

The backend uses `Socket.io` for real-time email notifications.

Event	Direction	Description
<code>subscribe</code>	Client → Server	Join room for an email address
<code>unsubscribe</code>	Client → Server	Leave room for an email address
<code>newEmail</code>	Server → Client	New email received notification

Connection: `javascript`

```
const socket = io('http://localhost:3000');
socket.emit('subscribe', 'abcd7k2m9x@tempmail.dev');
socket.on('newEmail', (message) => { /* handle new email */ });
```

SMTP Server

- Listens on port defined by `SMTP_PORT` (default: 2525)

- Only accepts emails for the configured `MAIL_DOMAIN`

- Auth is optional (open relay for receiving)
- Parses incoming emails and stores them in the `inbox` collection
- Notifies connected `WebSocket` clients in real-time

Cleanup Job

Runs every 5 minutes and:

- Finds all emails where `expires_at < NOW()` and `status = ASSIGNED`

- Marks them as EXPIRED

- Deletes their inbox messages from the database

Important: Each email has its own `expires_at` timestamp (1 hour from creation). Only emails that have actually expired are affected – other users' emails remain untouched.

Database Schema

Emails Collection/Table

Field	Type	Description
<code>email_address</code>	String (unique)	The temporary email address
<code>status</code>	String	AVAILABLE, ASSIGNED, or EXPIRED
<code>session_id</code>	String	The session that owns this email
<code>created_at</code>	Timestamp	When the email was created
<code>expires_at</code>	Timestamp	When the email will expire

Inbox Collection/Table

Field	Type	Description
<code>email_address</code>	String	Recipient email address
<code>sender</code>	String	Sender's email address
<code>subject</code>	String	Email subject line
<code>body</code>	String	Email body content
<code>received_at</code>	Timestamp	When the email was received

Setup & Running

Prerequisites

- Node.js 18+
- MongoDB Atlas account (free tier) OR PostgreSQL

Installation

```
bash
cd TempMailProj
npm install
```

Configuration

- Copy `.env` and fill in your database credentials
- Set `DB_TYPE` to `mongodb` or `postgres`

Run Development

```
bash
npm run dev
```

Run Production

```
bash
npm start
```

Error Handling

All endpoints return errors in the format:

```
json
{
  "error": "Description of the error"
}
```

Status Code	Meaning
400	Bad Request - Missing required parameters
404	Not Found - Resource doesn't exist
429	Too Many Requests - Rate limit exceeded
500	Internal Server Error

Security Features

- CORS configured for specific frontend origin
- Rate limiting (per IP and per session)
- Input sanitization (string trimming)
- Request body size limited to 10KB
- SMTP only accepts mail for configured domain

- Email uniqueness enforced via database indexes